Session 19 Homework — A Third Resource, a Deeper Query, and Two Real Design Questions

CORE, STRETCH, and CHALLENGE are all required — every one of them must be completed. The three names describe increasing difficulty only, nothing about which parts you submit.

Context

By the end of class, `schema.prisma` describes `User` / `Workspace` / `WorkspaceMember` / `Project`, each migrated for real against a real PostgreSQL database, and one route proves a real, multi-level relation query (`include`). Nobody has asked you yet to add a model whose own delete behavior is nullable (SET NULL, not CASCADE), or to reason in writing about what this session deliberately left out.

Goal

An extended `schema.prisma` with a real `Issue` model, migrated with zero data loss and a real, proven SET NULL delete behavior (CORE); a real route proving a three-level relation traversal (STRETCH); and two short, specific, written answers addressing what this session deliberately deferred (CHALLENGE).

What to build

You already know how to add a model, a relation, a delete-behavior choice, `prisma migrate dev`, and `include`. Research nothing new — apply what you already have:

- `Issue`: references a project (`project_id`), has a nullable assignee and reporter (both real references to `User`), a `title`, a `status`. Match `DATABASE-PROGRESSION.md`'s own frozen delete-behavior categorization exactly — you already know which category each of these three foreign keys belongs to from Block 4. This is the **bounded Session 19 teaching subset** of `Issue`, not the complete frozen entity — `key` and `Project`'s own `issue_seq` are explicitly deferred to Session 20; `description` / `priority` / `storyPoints` / `sprintId` / `updatedAt` are deferred to whichever later slice actually needs them. Do not add any of those now.
- A real proof that deleting a user who is assigned to and/or reported a real issue sets both foreign keys to `NULL` on that real issue row, without deleting the issue itself.
- A real route returning a workspace together with its projects, each project's own issues, and each issue's own assignee/reporter — one level deeper than teamwork's own `include` nesting.
- Two short, written answers (see CHALLENGE).

CORE (required)

Add `Issue` to `schema.prisma`, migrate it, and prove the real SET NULL behavior (create a real issue with a real assignee/reporter, delete that user, query the issue again).

STRETCH (required)

A new route on `PrismaCheckController` (e.g. `GET /prisma-check/workspaces/:slug/full`) returning a real workspace → its projects → each project's own issues → each issue's own assignee/reporter, all in one query, proven against real seeded data.

CHALLENGE (required)

Write two short, direct, written answers (as code comments above your `Issue` model, or in your own short notes):

1. `Issue` has no `key` field and `Project` has no `issue_seq` column here, even though Block 4's own `sql-practice/schema.sql` already implements exactly that by hand with a real transaction. Why would simply adding a plain, non-atomic counter field to `Project` and incrementing it from application code (read it, add one, save it, all as separate steps) be unsafe under concurrent requests? Name the actual race condition.
2. This session's new PostgreSQL database started completely empty — no real MongoDB document was copied into it. Before any existing route could genuinely be switched from Mongoose to Prisma without losing data, what would need to happen first, in order? Name at least three concrete steps.

Rules

Allowed: everything already shown live or in teamwork — `model`, scalar fields, `@id` / `@default` / `@unique` / `@map` / `@@map`, `@relation` / `onDelete`, `prisma migrate dev`, `PrismaService` / `PrismaModule`, `include` (any depth).

Not yet: no Prisma transaction/raw-SQL-interop syntax, no real issue-key counter generation in code — CHALLENGE question 1 asks you to explain why, in writing, not to build it anyway.

Test it

BASH

```
# inside api/
npx prisma migrate dev --name add_issues
npm run start
```

BASH

```
# in a second terminal
curl http://localhost:3000/prisma-check/workspaces/<your-real-slug>/full
```

Create a real test user, assign/report a real issue to them, delete that user, then re-query the issue directly (`npx prisma studio`, or a short script) to confirm both foreign keys read `null`.

Done when

- `Issue` exists, migrated, with no `DROP` / `ALTER COLUMN` in its own migration file.

- The real SET NULL proof actually ran, with a real before/after result, not assumed.
- The STRETCH route genuinely nests three levels and returns real data for a real seeded workspace.
- Both CHALLENGE answers name a concrete mechanism (a specific race condition; a specific, ordered migration step list), not a restatement of the question or a generic "ORMs are good" answer.

Working time

About 30–40 minutes — the model/migration work directly repeats what teamwork already proved; the two written answers deserve genuine thinking time.